Programação Orientada a Objetos em Java.

Assuntos abordados

- *Conceitos de programação orientada a objetos existentes na sintaxe da linguagem Java*
  - *Artefatos: pacote, classe, objeto, membro, atributo, método, construtor e interface*
  - *Características OO em Java: abstração, encapsulamento, herança e polimorfismo*
- *Sintaxe Java para construção de estruturas de dados*
  - *Tipos de dados primitivos*
  - *Componentes de uma classe*
- *Construção de aplicações simples em Java*
  - *Como construir uma classe Java (um tipo de dados) contendo métodos, atributos e construtores*
  - *Como construir e usar objetos*
- *Este módulo é longo e aborda muitos assuntos que serão tratados novamente em módulos posteriores*

Por que OO é importante?

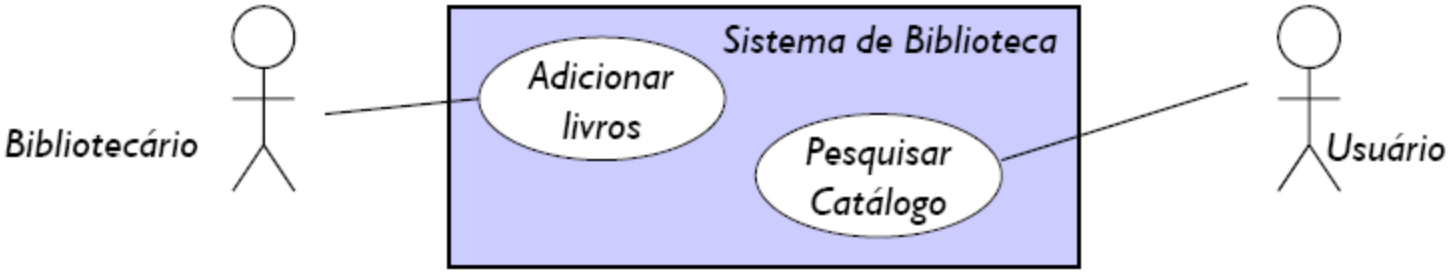
- *Java é uma linguagem orientada a objetos*
- *Para desenvolver aplicações e componentes de qualidade em Java é preciso entender e saber aplicar princípios de orientação a objetos ao programar*
- *É possível escrever programas em Java sem saber usar os recursos da OO, mas*
  - *Dificilmente você será capaz de ir além de programas simples com mais de uma classe*
  - *Será muito difícil entender outros programas*
  - *Seu código será feio, difícil de depurar e de reutilizar*
  - *Você estará perdendo ao usar uma linguagem como Java*

(se quiser implementar apenas rotinas procedurais pode usar uma linguagem melhor para a tarefa como Shell, Fortran, etc.)

O que é orientação a objetos

- *Paradigma do momento na engenharia de software*
  - *Afeta análise, projeto (design) e programação*
- *A análise orientada a objetos*
  - *Determina o que o sistema deve fazer: Quais os atores envolvidos? Quais as atividades a serem realizadas?*
  - *Decompõe o sistema em objetos: Quais são? Que tarefas cada objeto terá que fazer?*
- *O design orientado a objetos*
  - *Define como o sistema será implementado*
  - *Modela os relacionamentos entre os objetos e atores (pode-se usar uma linguagem específica como UML)*
  - *Utiliza e reutiliza abstrações como classes, objetos, funções, frameworks, APIs, padrões de projeto*

Análise OO e Procedural

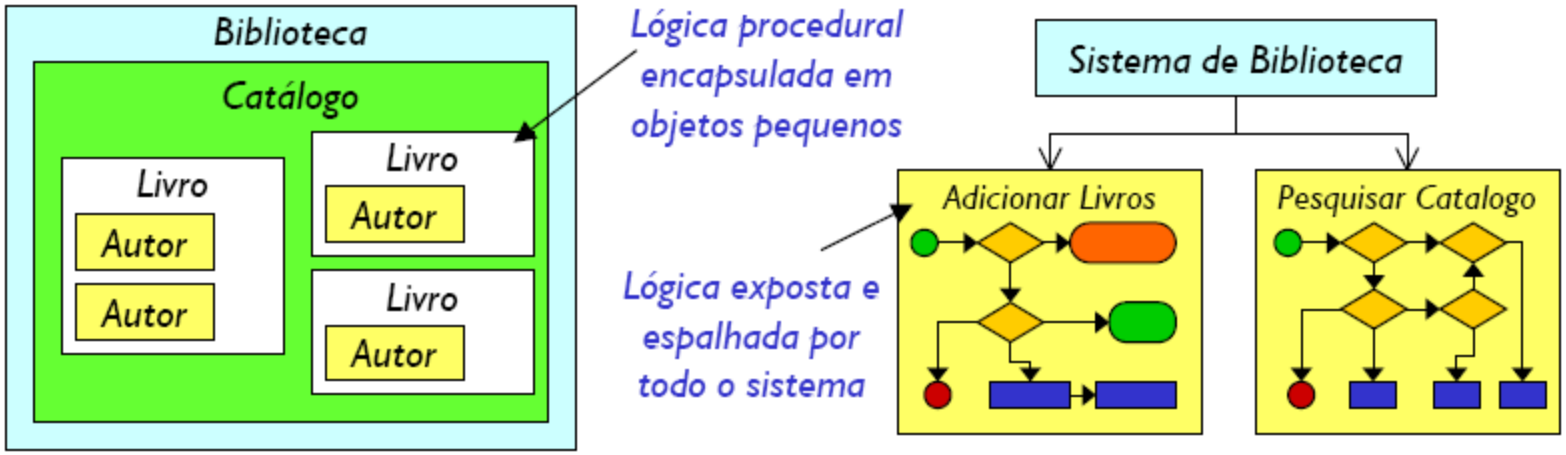


(1) Trabalha no **espaço do problema** (casos de uso simplificados em objetos)

- Abstrações mais simples e mais próximas do mundo real

(2) Trabalha no **espaço da solução** (casos de uso decompostos em procedimentos algorítmicos)

- Abstrações mais próximas do mundo do computador



O que é um objeto?

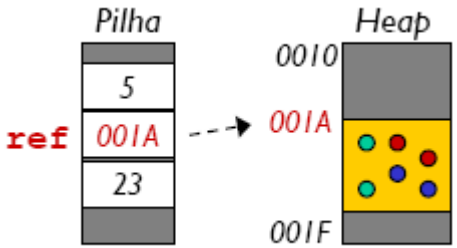
- *Objetos são conceitos que têm*
  - *identidade,*
  - *estado e*
  - *Comportamento*
- *Características de Smalltalk, resumidas por Allan Kay:*
  - *Tudo (em um programa OO) são objetos*
  - *Um programa é um monte de objetos enviando mensagens uns aos outros*
  - *O espaço (na memória) ocupado por um objeto consiste de outros objetos*
  - *Todo objeto possui um tipo (que descreve seus dados)*
  - *Objetos de um determinado tipo podem receber as mesmas mensagens*

Em outras palavras...

- *Em uma linguagem OO pura*
  - *Uma variável é um objeto*
  - *Um programa é um objeto*
  - *Um procedimento é um objeto*
- *Um objeto é composto de objetos, portanto*
  - *Um programa (objeto) pode ter variáveis (objetos que representam seu estado) e procedimentos (objetos que representam seu comportamento)*
- *Analogia: abstração de um telefone celular*
  - *É composto de outros objetos, entre eles bateria e botões*
  - *A bateria é um objeto também, que possui pelo menos um outro objeto: carga, que representa seu estado*
  - *Os botões implementam comportamentos*

Objetos

- *Em uma linguagem orientada a objetos pura*
  - *Um número, uma letra, uma palavra, uma valor booleano, uma data, um registro, uma botão da GUI são objetos*
- *Em Java, objetos são armazenados na memória de heap e manipulados através de uma referência (variável), guardada na pilha.*
  - *Têm estado (seus atributos)*
  - *Têm comportamento (seus métodos)*
  - *Têm identidade (a referência)*



- *Valores unidimensionais não são objetos em Java*
  - *Números, booleanos, caracteres são armazenados na pilha*
  - *Têm apenas identidade (nome da variável) e estado (valor literal armazenado na variável); - dinâmicos; + rápidos*

Variáveis, valores e referências

- Variáveis são usadas em linguagens em geral para armazenar valores
- Em Java, variáveis podem armazenar endereços de memória do heap ou valores atômicos de tamanho fixo
  - Endereços de memória (referências) são inaccessíveis aos programadores (Java não suporta aritmética de ponteiros!)
  - Valores atômicos representam tipos de dados primitivos
- Valores são passados para variáveis através de operações de atribuição
  - Atribuição de valores é feita através de literais
  - Atribuição de referências (endereços para valores) é feita através de operações de construção de objetos e, em dois casos, pode ser feita através de literais

Literais e tipos

- *Tipos representam um valor, uma coleção de valores ou coleção de outros tipos. Podem ser*
  - *Tipos básicos, ou primitivos, quando representam unidades indivisíveis de informação de tamanho fixo*
  - *Tipos complexos, quando representam informações que podem ser decompostas em tipos menores. Os tipos menores podem ser primitivos ou outros tipos complexos*
- *Literais: são valores representáveis literalmente.*
  - *Números:* `1, 3.14, 1.6e-23`
  - *Valores booleanos:* `true` e `false`
  - *Caracteres individuais:* `'a', '\u0041', '\n'`)
  - *Seqüências de caracteres:* `"aaa", "Java"`
  - *Vetores de números, booleanos ou strings:* `{"a", "b"}`

Unidimensionais

Compostos

Tipos primitivos e complexos

- *Exemplos de tipos primitivos (atômicos)*
  - *Um inteiro ou um caractere,*
  - *Um literal booleano (true ou false)*
- *Exemplos de tipos complexos*
  - *Uma data: pode ser decomposta em três inteiros, representando dia, mês e ano*
  - *Um vetor de inteiros: pode ser decomposto em suas partes*
  - *Uma seqüência de caracteres: pode ser decomposta nos caracteres que a formam*
- *Em Java, tipos complexos são armazenados como objetos e tipos primitivos são guardados na pilha*
  - *Apesar de serem objetos, seqüências de caracteres (strings) em Java podem ser representadas literalmente.*

Tipos primitivos em Java

- *Têm tamanho fixo. Têm sempre valor default.*
- *Armazenados na pilha (acesso rápido)*
- *Não são objetos. Classe 'wrapper ' faz transformação, se necessário (encapsula valor em um objeto).*

Tipo	Tamanho	Mínimo	Máximo	Default	'Wrapper'
boolean	—	—	—	false	Boolean
char	16-bit	Unicode 0	Unicode $2^{16} - 1$	\u0000	Character
byte	8-bit	-128	+127	0	Byte
short	16-bit	-2^{15}	$+2^{15} - 1$	0	Short
int	32-bit	-2^{31}	$+2^{31} - 1$	0	Integer
long	64-bit	-2^{63}	$+2^{63} - 1$	0	Long
float	32-bit	IEEE754	IEEE754	0.0	Float
double	64-bit	IEEE754	IEEE754	0.0	Double
void	—	—	—	—	Void

Exemplos de tipos primitivos e literais

- *Literais de caracter:*

```
char c = 'a';
char z = '\u0041'; // em Unicode
```

- *Literais inteiros*

```
int i = 10;  short s = 15;  byte b = 1;
long hexa = 0x9af0L; int octal = 0633;
```

- *Literais de ponto-flutuante*

```
float f = 123.0f;
double d = 12.3;
double g = .1e-23;
```

- *Literais booleanos*

```
boolean v = true;
boolean f = false;
```

- *Literais de string (não é tipo primitivo - s é uma referência)*

```
String s = "abcde";
```

- *Literais de vetor (não é tipo primitivo - v é uma referência)*

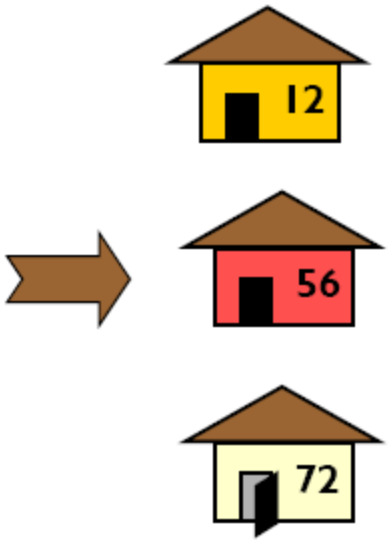
```
int[] v = {5, 6};
```

O que é uma classe?

- Classes são uma especificação para objetos
- Uma classe representa um tipo de dados complexo
- Classes descrevem
  - Tipos dos dados que compõem o objeto (o que podem armazenar)
  - Procedimentos que o objeto pode executar (o que podem fazer)

Classe

Instâncias da classe Casa (objetos)




```
Casa c1 = new Casa();
c1.numero = 12;
c1.cor = Color.yellow;

Casa c2 = new Casa();
c2.numero = 56;
c2.cor = Color.red;

Casa c3 = new Casa();
c3.numero = 72;
c3.cor = Color.white;
c3.abrePorta();
```

Continuando...

- *Classes não são os objetos que representam*
  - *A planta de uma casa é um objeto, mas não é uma casa*
- *Classes definem lógica estática*
  - *Relacionamentos entre classes são definidos na programação e não mudam durante a execução*
  - *Relacionamentos entre objetos são dinâmicos e podem mudar. O funcionamento da aplicação reflete a lógica de relacionamento entre os objetos, e não entre as classes.*
- *Classes não existem no contexto da execução*
  - *Uma classe representa vários objetos que ocupam espaço na memória, mas ela não existe nesse domínio*
  - *A classe tem papel na criação dos objetos, mas não existe quando os objetos trocam mensagens entre si.*

Resumindo...

- *Objetos são conceitos que têm estado (atributos), comportamento (métodos) e identidade (referência)*
- *Tipos representam valores*
  - *Primitivos: valores fixos e indivisíveis. São armazenados na pilha*
  - *Complexos: valores multidimensionais que podem ser decompostos em componentes menores. Descrevem objetos que são armazenados no heap*
- *Literais*
  - *Usados para definir tipos primitivos ou certos tipos complexos formados por componentes iguais (strings e vetores)*
- *Variáveis podem armazenar valores de tipos primitivos ou referências para objetos*
- *Classes são tipos complexos: descrevem objetos*
  - *Não são importantes no contexto da execução*

Atributos e métodos

- *Uma classe define uma estrutura de dados não-ordenada*
  - *Pode conter componentes em qualquer ordem*
- *Os componentes de uma classe são seus membros*
- *Uma classe pode conter três tipos de componentes*
  - *Membros estáticos ou de classe: não fazem parte do "tipo"*
  - *Membros de instância: definem o tipo de um objeto*
  - *Procedimentos de inicialização*
- *Membros estáticos ou de classe*
  - *Podem ser usados através da classe mesmo quando não há objetos*
  - *Não se replicam quando novos objetos são criados*
- *Membros de instância*
  - *Cada objeto, quando criado, aloca espaço para eles*
  - *Só podem ser usados através de objetos*
- *Procedimentos de inicialização*
  - *Usados para inicializar objetos ou classes*

```
public class Casa {

    private Porta porta;
    private int numero;
    public java.awt.Color cor;

    public Casa() {
        porta = new Porta();
        numero = ++contagem * 10;
    }

    public void abrePorta() {
        porta.abre();
    }

    public static String arquiteto = "Zé";
    private static int contagem = 0;

    static {
        if ( condição ) {
            arquiteto = "Og";
        }
    }
}
```

Atributos de instância: cada objeto poderá armazenar valores diferentes nessas variáveis.

*Procedimento de inicialização de **objetos** (Construtor): código é executado após a criação de cada novo objeto. Cada objeto terá um número diferente.*

Método de instância: só é possível chamá-lo se for através de um objeto.

Atributos estáticos: não é preciso criar objetos para usá-los. Todos os objetos os compartilham.

*Procedimento de inicialização **estático**: código é executado uma única vez, quando a classe é carregada. O arquiteto será um só para todas as casas: ou Zé ou Og.*

Boas práticas ao escrever classes

- *Use, e abuse, dos espaços*
  - *Endente, com um tab ou 4 espaços, membros da classe,*
  - *Endente com 2 tabs, o conteúdo dos membros, ...*
- *A ordem dos membros não é importante, mas seguir convenções melhora a legibilidade do código*
  - *Mantenha os membros do mesmo tipo juntos (não misture métodos estáticos com métodos de instância)*
  - *Declare as variáveis antes ou depois dos métodos (não misture métodos, construtores e variáveis)*
  - *Mantenha os seus construtores juntos, de preferência bem no início*
  - *Se for necessário definir blocos static, defina apenas um, e coloque-o no início ou no final da classe.*

Construtores

- *Construtores são procedimentos realizados na construção de objetos*
  - *Parecem métodos, mas não têm tipo de retorno e têm nome idêntico ao nome da classe*
  - *Não fazem parte da definição do tipo do objeto (interface)*
  - *Nem sempre aparecem explícitos em uma classe: podem ser omitidos (o sistema oferece uma implementação default)*
- *Para cada objeto, o construtor é chamado exatamente uma vez: na sua criação*
  - *Exemplo:*

```
> Objeto obj = new Objeto();
```

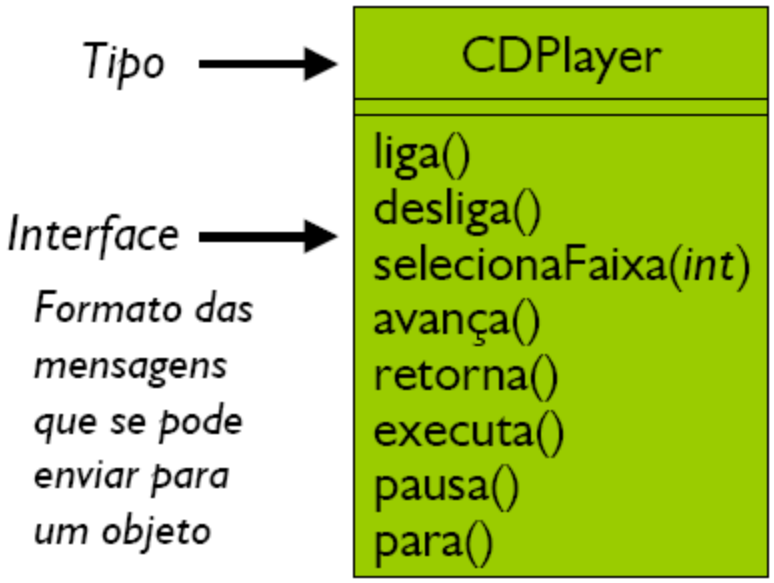
  - *Alguns podem requerer parâmetros*

```
> Objeto obj = new Objeto(35, "Nome");
```

Chamada de construtor

Objetos possuem uma interface

- Através da interface* é possível utilizá-lo
  - Não é preciso saber dos detalhes da implementação
- O tipo (Classe) de um objeto determina sua interface
  - O tipo determina quais mensagens podem ser enviadas



Em Java

```
(...)
CDPlayer cd1;
cd1 = new CDPlayer();
cd1.liga();
cd1.selecionaFaixa(3);
cd1.executa();
(...)
```

Classe Java (tipo)

Referência

Criação de objeto

Envio de mensagem

* interface aqui refere-se a um conceito e não a um tipo de classe Java

E uma implementação (oculta)

- *Implementação não interessa à quem **usa** **objetos***
- *Papel do usuário de classes*
  - *não precisa saber como a classe foi escrita, apenas quais seus métodos, quais os parâmetros (quantidade, ordem e tipo) e valores que são retornados*
  - *usa apenas a interface (pública) da classe*
- *Papel do desenvolvedor de classes*
  - *define novos tipos de dados*
  - *expõe, através de métodos, todas as funções necessárias ao usuário de classes, e oculta o resto da implementação*
  - *tem a liberdade de mudar a implementação das classes que cria sem que isto comprometa as aplicações desenvolvidas pelo usuário de classes*

Resumindo

- *Os componentes de uma classe, em Java, podem pertencer a dois domínios, que determinam como são usados*
  - *Domínio da classe: existem independentemente de existirem objetos ou não: métodos static, blocos static, atributos static e interface dos construtores de objetos*
  - *Domínio do objeto: métodos e atributos não declarados como static (definem o tipo ou interface que um objeto possui), e conteúdo dos construtores*
- *Construtores são usados apenas para construir objetos*
  - *Não são métodos (não declaram tipo de retorno)*
  - *"Ponte" entre dois domínios: são chamados uma vez antes do objeto existir (domínio da classe) e executados no domínio do objeto criado*
- *Separação de interface e implementação*
  - *Usuários de classes vêem apenas a interface.*
  - *Implementação é encapsulada dentro dos métodos, e pode variar sem afetar classes que usam os objetos*

Referências

- Adaptado de Helder da Rocha, **J100: Programação Orientada a Objetos usando Java**. www.argonavis.com.br
- Java como programar, 6 ed.